אוניברסיטת בן-גוריון בנגב
Ben-Gurion University of the Negev

Course ID - 202.1.4481

March 2026

Ranking Files For Exfiltration Using Machine Learning

Network Security Project Report

Project by Ilan Rozinko (ID 213945934)

ATTACHED GITHUB REPOSITORY: →
crasheded/ML-Powered_File_Prioritization_Engine VIEW REPO

Table of contents:

1. Introduction and Abstract ..... **Page 3**

- **1.1.** Background
- **1.2.** Goals
- **1.3.** Our Approach
- **1.4.** Results

2. Dataset Creation & Preprocessing ..... **Page 6**

- **2.1.** Data Sourcing
- **2.2.** Our Data Scheme
- **2.3.** Data Augmentation & Modernization
- **2.4.** Data Labeling Logic
- **2.5.** Dataset Distribution and Handling Class Imbalance

3. Feature Engineering ..... **Page 12**

- **3.1.** Metadata Heuristics
- **3.2.** Temporal Analysis - Recency Score
- **3.3.** Structural Text Hashing

4. Model Selection and Training ..... **Page 15**

- **4.1.** The Logistic Regression Model Explained
- **4.2.** Why Logistic Regression?
- **4.3.** Target-Specific Training
- **4.4.** Regularization and Metadata Stability
- **4.5.** Addressing Class Imbalance via Penalty Weights
- **4.6.** Prioritizing Recall over Nominal Accuracy

5. System Architecture & C++ Implementation ..... **Page 17**

- **5.1.** The Python-to-C++ Bridge
- **5.2.** Traversing the Filesystem
- **5.3.** Extracting Metadata on the Fly
- **5.4.** Feature Engineering in C++
- **5.5.** Inference and Scoring

6. Results & Evaluation ..... **Page 19**

- **6.1.** Model Performance
- **6.2.** Qualitative Results
- **6.3.** System Performance

7. Obstacles, Struggles & Engineering Challenges ..... **Page 22**

8. Future Work & Improvements ..... **Page 23**

9. Conclusion ..... **Page 24**

1. Introduction and Abstract:

1.1. Background:

In modern computer networks, cyberattacks designed for data exfiltration face significant constraints regarding the volume and total size of the data they can move. These limitations are often necessary to ensure payload stealth and to bypass network monitoring systems during the transfer phase back to the attacker's infrastructure or they may be related to the implementation of the malware itself.

To operate effectively within these constraints, malware developers must devise intelligent prioritization methods. The goal is to distinguish between high-value, sensitive files and the vast amount of "junk" files present on a network. If implemented effectively an attacker can maximize the impact of a breach while minimizing the risk of detection.

Classic approaches include basic keyword matching and extension filtering. For example, many older scripts simply look for the word "password" in a filename or gather every .xlsx file they can find. However, these methods have significant drawbacks:

- **Performance Overhead:**

  Scanning the actual content of every file is slow and heavy on CPU usage. In a real-world attack, this behavior makes it easy for antivirus software systems to spot the suspicious activity.

- **Low Precision:**

  Simply looking for extensions like .pdf or .docx is often too broad. A modern workstation or network might contain thousands of irrelevant documents, making it impossible to find the truly important ones within a limited time window. High precision requires prior knowledge of the HVTs(high Value Targets) and target network.

To solve this, our project shifts the focus from file content to file metadata and directory structure. Using **machine learning** to analyze how a file is named, where it is located in the folder hierarchy, its modification history and other parameters, we can predict its value with a high degree of accuracy. This allows the system to remain lightweight, fast, and covert – while identifying the most sensitive targets without ever needing to open a single file.

1.2. Goals:

To build a system capable of identifying and classifying sensitive files across a filesystem, which will be able to be used as part of malware that is designed to exfiltrate high priority files from breached computer networks.

Our primary objectives are:

- **Multi-Target Classification:**
  The system is designed to be flexible, supporting multiple "focuses" rather than a generic approach. For this project, we designated **Finance**, **HR**, and **IT** as our primary targets. The classification logic is built to prioritize files for each focus separately, allowing the system to surface the most relevant documents for a specific target type while filtering out the rest.

- **Accuracy in Noise:**
  The system must be able to distinguish "signal" (sensitive data) from "noise" (system files, application data, and junk) in a highly imbalanced environment where sensitive files are statistically rare.

- **Efficiency:**
  The system must be efficient in its use of compute resources as well as having a low dwell time - in order to improve the stealth of the payload.

1.3. Our Approach:

Our proposed solution is a **file metadata-centric machine learning model**. Given the metadata of the file (such as filename, file path, modification date, etc.) our trained models output a score between 0 and 1. We trained 4 separate models, One for each of the previously mentioned focuses.

Scores approaching 1 indicate a high-value file, such as a financial report or HR record, while a score near 0 indicates "junk" or non-essential system files.

Our implementation has two distinct components:

- **ML Model Training Module (Python):**
  Using feature engineering this module converts raw file metadata from our training dataset into numerical features. These numerical features are then used to train 4 Logistic Regression classifiers using *scikit-learn* - one for each focus (Finance, HR, IT, General).

  The resulting models learn to distinguish high value sensitive files from irrelevant files and the vast amount of non-essential system "noise". The trained model weights are then used in the actual Payload.

- **Payload (C++):**
  The payload is a lightweight, standalone executable that applies the trained models to a real filesystem. It incorporates the learned weights and classification logic into a standalone binary, enabling real-time inference without requiring a Python environment – simulating real world use. The component traverses the target drive, extracts metadata on the fly, and generates a probability score for each file per focus, allowing for the immediate ranking and prioritization of sensitive assets.

Another major component of the project is the **construction of the training dataset**. As there was no readily available dataset that fit our requirements we constructed and labeled a custom dataset using various sources, such as corporate filesystem snapshots, GitHub repositories, public US government documents, as well as purely synthetic and LLM generated data.

1.4. Results:

The system successfully fulfilled its primary objectives, identifying sensitive files within complex directory structures and achieving **high recall across all specialized categories** (ranging from 0.82 for IT to 0.89 for Finance) despite their statistical rarity in the dataset. By focusing on structural patterns, textual contents and temporal data, the model effectively filters out most system "noise", as well as prioritizing recent files over outdated assets.

The C++ payload confirmed the project's efficiency goals, delivering **sub-millisecond inference** speeds per file while maintaining a negligible CPU footprint and minimal dwell time. It is also entirely self-contained within its binary and requires no external libraries or binaries.

These results prove that a **metadata-centric approach is a viable and stealthy alternative to deep-content scanning**, providing rapid, real-time prioritization across millions of files for real-world deployment.

Figure 1: *Snippet from the Payload - Ranking according to the Finance model.*

```
ANALYZING TARGET: FINANCE

--- TOP 10 FILES ---
Score: 0.9469 | D:\MLDATA\Users\hubbardkim\OneDrive - Corp\Finance\Forecasts\FY2023_Corp_Budget_v6.xlsx
Score: 0.9368 | D:\MLDATA\Users\hubbardkim\OneDrive -Corp\Finance\Forecasts\Forecast_Model_FINAL.xlsm
Score: 0.9205 | D:\MLDATA\Users\kmorris\Documents\Finance\Reporting\HR_Forecast_Sep2024.xlsx
Score: 0.9134 | D:\MLDATA\Users\goodjeffrey\Documents\Finance\Reporting\Ops_Forecast_May2025.xlsx
Score: 0.8889 | D:\MLDATA\Users\ybrown\Documents\Finance\Reporting\R&D_Forecast_Sep2024 (1).xlsx
Score: 0.8810 | D:\MLDATA\Users\arthur13\Documents\Finance\Reporting\Regional_Forecast_Sep2024.xlsx
Score: 0.8384 | D:\MLDATA\Finance\Temp\HR_Forecast_Feb2023.xlsx
Score: 0.8283 | D:\MLDATA\Shared\Finance\Uploads\HR_Forecast_Apr2025.xlsx
Score: 0.8168 | D:\MLDATA\Finance\Temp\Corp_Forecast_Feb2024.xlsx
Score: 0.8061 | D:\MLDATA\Users\patricia94\OneDrive - Corp\Finance\Forecasts\FY2024_Sales_Budget_v5.xlsx
```

2. Dataset Creation & Preprocessing

Because no single dataset exists that fully captures the metadata landscape of a modern corporate computer network, we constructed a multi-layered custom dataset for our project. This involved merging real-world publicly available data with targeted synthetic records. We performed various data augmentations and modernizations and implemented a custom data labeling system for each focus. This comprehensive process ensures the model can accurately distinguish between mundane system "noise" and high-value sensitive assets.

2.1. Data Sourcing:

The foundation of our dataset relies on three primary sources of "real-world" file structures to provide a baseline of realism:

- **M57-Patent Dataset:** This forensic corpus provides thousands of authentic file entries from a simulated corporate environment, including real user-generated folder structures and files as well as system files. We converted publicly available disk images into a usable table format.

- **GovDocs Database:** To strengthen the "Finance" and "HR" model, we incorporated metadata from public US government documents. These provide the model with the naming conventions and file-type distributions typical of official reports, spreadsheets, and records.

- **GitHub Repositories:** To build a robust "IT" focus, we utilized metadata from diverse software projects. This gave the model exposure to source code hierarchies, build artifacts, and development environments that are common on modern workstations and realistic Dev environments.

To simulate the "feel" of a real filesystem, we programmatically injected several categories of **"synthetic noise"** designed to challenge the model's precision, and allow it to generalize better to the real world:

- **System and Dependency Noise:**
  We added records representing the deep, repetitive directory structures of operating systems and development frameworks (e.g., node_modules, site-packages, or Windows System32 subdirectories).

- **Structural Obfuscation:**
  We generated "long-name" files—metadata entries with extremely long paths or filenames—to mimic logs or obfuscated files that often confuse simple keyword-based scanners.

- **IT "Secret Files" Signal:**
  We manually injected rows representing critical IT assets that are statistically rare and are not typically found on github, such as private keys (.pem, .key), encrypted databases (.kdbx), and environment configurations files (.env). These rows ensure the model learns that certain small, buried files are HVTs.

The most difficult data to source in high volume is **realistic records and documents** relevant for HR and Finance models. To address this, we utilized LLMs to generate synthetic but realistic metadata records, based on actual records found in M57 and GovDocs datasets. This allowed us to extrapolate the existing records into further high signal rows available to the models.

2.2. Our Data Scheme:

The following data scheme defines the fundamental metadata attributes used to represent each record within the training dataset and the actual system traversal performed in the Payload:

Column Name	Data Type	Description	Examples
filename	String	The name of the file	budget_2025.xlsx privkey.pem mscorld.dll
path	String	The full directory path	Z:\HR\Recruiting\Reqs\2024\Profiles C:\Users\martin05\source\repos\ctags C:\Program Files\AVG\AVG9
extension	String	The file type	.pdf, .docx, .xlsx, .py, .png, .key, .pem, .sql
size_bytes	Integer	The physical size of the file on the disk in bytes	2249 bytes
mod_time_unix	Integer	The last modification timestamp in Unix format	1750637212 = 06:52 23/06/2025
source_type	Category	The origin of the data, used for labeling	GitHub_real, GovDocs_real, System_noise, ChatGPT_Finance, etc.

2.3. Data Augmentation & Modernization:

To bridge the gap between 2009 forensic data present in the M57 dataset, The unstructured nature of the GitHub and GovDocs datasets we had to perform various augmentations to fit the records into a usable format:

M57: Deep Architectural Modernization

- **Legacy-to-Modern Migration:**
  We re-mapped legacy paths (e.g., Documents and Settings) to modern standards (e.g., Users) and updated system locations to current Windows structures. We added OneDrive paths to some of the user documents and WSL paths to simulate modern developer workstations.

- **Software Injection:**
  In order to simulate a modern workspace, we programmatically injected directory structures for commonly used Applications like Slack, Microsoft Teams, Zoom, Docker, and VS Code into the "Program Files" paths.

GovDocs & GitHub:

- **Context-Aware Mapping**: Files were assigned to 22 corporate categories (e.g., *finance_core*, *hr_recruiting*) based on filename keywords and placed in matching directory structures.

- **IT Ecosystem Simulation**: Technical paths were generated for GitHub metadata, including local Windows repos (*C:\Users\{user}\source\repos*), Linux WSL paths *(\\wsl$\Ubuntu\home*), and production servers.

- **Persona-Based User Generation**: The Faker library generated hundreds of developer and employee usernames, including specialized roles like gitlab_runner and sysadmin, which were then used to construct corresponding file paths.

Other Augmentations:

- **Extension Upgrades**: Legacy binary extensions (.doc, .xls, .ppt) were converted to modern formats (.docx, .xlsx, .pptx).

- **Temporal Shifting**: We reset the timestamp of various records, ensuring that there's enough signal for the model to learn to appreciate the value of more recent files.

- **Size Normalization**: We adjusted file sizing where necessary, as well as performing other minor changes to increase our model performance.

2.4. Data Labeling logic:

Before the model can perform inference, the dataset must be labeled for each one of the focuses. This was performed via a complex, multi-stage labeling process. This complex process was used to define "high value" data across the four distinct focuses, moving far beyond simple keyword matching:

Multi-Target Focus:
To allow for target-specific optimization, the system utilizes four binary label columns: *label_finance* for economic records, *label_hr* for personnel data, *label_it* for infrastructure and technical secrets, and *label_general* as a catch-all for the other columns. Put simply, if a record was marked as a HVT in any other column *label_general* would also be marked as 1.

The labeling script employs strict rules to distinguish signal from noise:

- **Hard-coded Pattern Matching**:
  Finance and HR files require both a target keyword (e.g., "budget" or "resume") and a valid document extension (e.g., .pdf, .xlsx, .docx) to be labeled as a positive signal. The same principle applies to IT files.

- **Recency-Gated Signaling:**
  Labels are strictly tied to file recency. A file is only assigned a positive label if it matches a sensitive pattern and was modified within a defined recency window (365 days). Older files matching the same patterns are assigned a label of 0 to teach the model to prioritize recent, more relevant files.

- **Toxic Folder Exclusion:**
  The engine implements a strict "Toxic Filter" that overrides all other rules. Any file located within structural noise directories—such as *node_modules*, *.git*, *build*, or *__pycache__* is forced to a label of zero, regardless of keyword or path matches. This teaches the model to ignore IT files that it might deem valuable but turn out to be regular libraries or system files.

- **Source Integrity Filter:**
  To prevent false positives from noisy records, positive labeling is restricted to files originating from non-noise sources (GovDocs, GitHub repos, and LLM-generated signal).

2.5. Dataset Distribution and Handling Class Imbalance:

A significant challenge was the extreme statistical imbalance between "Signal" (sensitive files) and "Noise" (system and application files). In a typical filesystem, high-value targets such as financial records or HR documents represent a tiny fraction of total files, often representing less than 1% of total files. To ensure the model did not default to "junk" predictions to achieve high nominal accuracy, we took several steps to prepare the data for training.

The final training set was structured as a multi-layered dataset totaling 5,154 rows. We separate "Signal" from "Noise" across four specific focuses, and further categorize via recency filtering.

Master Dataset Distribution:

Category	Sub-Category	Sample Count	Target probability	Label	Description
Fresh Signal 786 rows	**Finance Focus**	182	3.5%	1	Recent budgets, invoices, and ledgers.
	HR Focus	266	5.2%	1	Recent resumes, offer letters, and payroll data.
	IT Focus	338	6.6%	1	Recent keys, certs, and config files.
	General Focus	786	15.2%	1	Recent files matching any HVT pattern(Finance, HR, or IT)
Stale Signal 355 rows	**Finance Focus**	86	1.7%	0	Files matching the previously mentioned HVT patterns yet possess old Timestamps.
	HR Focus	126	2.4%	0	
	IT Focus	143	2.8%	0	
	General Focus	355	6.8%	0	
Pure Noise 4013 rows 3206 fresh 807 stale	**Business Noise**	1,136	22.1%	0	Non-HVT documents from GovDocs.
	Dev Noise	1,136	22.1%	0	Generic GitHub code artifacts
	Dependency Noise	1,136	22.1%	0	Software dependecies (e.g., node_modules, .venv).
	System Noise	378	7.3%	0	Low priority OS and program files.
	Long-Name Noise	227	4.4%	0	Obfuscated and long filenames used for structural confusion.

Note: records may be labeled as 1 for multiple categories. Any records marked 1 in any of the three focuses will be marked 1 in General focus.

This structure ensures the model encounters "Signal" as a rare event, mimicking real-world filesystem distributions. By explicitly separating Fresh and Stale signals into different labels, the model learns that a file's sensitivity is a function of both its metadata and its recency.

2.6. Handling class imbalances via class weighting:

In the real world and as can be seen in our dataset, HVTs are a tiny fraction of the total amount of files present. To train the model effectively we employed **class weighting** during training.

Class weighting modifies the loss function by applying a multiplier to minority class errors, making them mathematically more "expensive" than majority errors, i.e. during training, predicting a HVT record wrongly is penalized harsher than making a wrong prediction for a noise records. This shifts the decision boundary to prioritize Recall, forcing the model to identify rare assets rather than defaulting to "noise" predictions to achieve high nominal accuracy.

Specific penalty multipliers were assigned: 12.0 for IT secrets, 4.0 for Finance/HR, and 2.0 for general. Regularization paramaters were tuned to prevent overfitting (further elaborated in the training section).

3. Feature Engineering

Feature engineering is the critical intermediary step that allows our model to perform effective predictions. By converting raw metadata into meaningful numerical features, we enable the system to differentiate between mundane system files and high-value assets. The following subsections detail the three categories of features utilized by the engine: **Metadata Heuristics**, **Temporal Analysis**, and **Structural Text Hashing**.

3.1. Metadata Heuristics:

This category includes features that can be easily interpolated from the dataset and converted to numerical features:

- **size_logged:** log1p of the file size in bytes - $ln(1 + size_bytes)$.

  Using a log scale prevents large files from disproportionately skewing the model's weights while maintaining the relative difference in scale.

- **name_len:** The character count of the filename.

  This helps the model distinguish between short, descriptive names and long, generated, or obfuscated filenames common in junk system files.

- **Path_len:** The total character count of the directory path.

  Longer paths often indicate files buried in deep application or system subdirectories, as humans don't tend to five long folder names.

- **Path_depth:** The number of directory levels in the file path, calculated by counting slashes in the file path string.

  This identifies how deep a file is nested within the folder hierarchy. Deep paths tend to indicate system and program paths, rather than human created directories that tend to be shallower.

- **valuable_ext:** A binary indicator (0 or 1) that flags if the file extension exists in a target-specific list of high-value formats (e.g., .xlsx for Finance or .pem for IT).

- **junk_ext:** similarly, A binary indicator (0 or 1) that flags if the file extension belongs to a known "noise" category, such as temporary logs, caches, or compiled code artifacts. We also check whether the file extension is relevant to the focus (e.g., .py would be marked as irrelevant for the HR model).

3.2. Temporal analysis - Recency score:

Time is one of the most reliable indicators of file relevance. In the context of identifying high-value targets, a financial spreadsheet modified last week is significantly more relevant than one last updated 10 years ago.

The score is calculated using a hyperbolic decay function. We chose this scheme because it provides a smooth, predictable drop-off in value that never reaches zero, ensuring that even older files retain a baseline level of "signal" for the model. The formula used in our implementation is:

$$\text{Recency Score} = \frac{1}{\frac{T_{ref} - T_{mod}}{C} + 1}$$

Where we define:

- $\boldsymbol{T_{ref}}$ **- Reference Date:** A fixed epoch representing "the present" (set to January 1, 2026, for this project).
- $\boldsymbol{T_{mod}}$ **- Modification Time:** The Unix timestamp of the file.
- $\boldsymbol{C}$ **-** The number of seconds in one year $(365 * 86400)$

By dividing the age of the file by the number of seconds in a year, we convert the "age" into a unit of years. This creates a "Half-Life" effect where a file exactly one year old receives a score of exactly 0.5. As the file gets older, the denominator grows, causing the score to diminish.

Practical Examples:

File Age	Calculation (Age in years)	Recency Score
1 Week	$1/\left(\frac{7}{365}+1\right)$	0.981
2 Years	$1/(2+1)$	0.333
10 Years	$1/(10+1)$	0.091

Why Use this Function?

Our hyperbolic approach ensures that the model sees a massive difference between a 1-day-old file and a 1-year-old file, but a much smaller difference between a 10-year-old file and an 11-year-old file. This mimics real-world logic, the "drop-off" in importance happens quickly at first and then levels out as the data becomes outdated or archival.

3.3. Structural Text Hashing:

To classify files effectively, the model must "read" the file path and name. However, standard machine learning models cannot process raw text - they require numerical input. While traditional methods like keyword matching (looking for "password") are common, they are easily bypassed by slight misspellings or varying folder structures.

To solve this, we use a **character-level N-grams** and a **"Hashing Trick":**

Instead of looking at whole words, we break every string into overlapping 4-character chunks called quadgrams. This allows the model to identify meaningful sub-patterns even if they are part of a larger, unfamiliar string. For example, the word "budget" is broken down into *budg*, *udge*, *dget*.

Once we have these quadgrams, we need to map them to a fixed-size numerical vector (in our case 2048 features). We use the MurmurHash3 algorithm to turn each quadgram into an integer index using modulo. If the quadgram *"budg"* hashes to index 402, we place a "1" in the 402nd position of our representative vector.

Hashing Example:

To understand how a raw path becomes a model input, provided is a toy example of hashing a short path into a 10-feature vector using quadgrams:

- **Input:** "work\IT"
- **Step 1 – extract quadgrams:** *"work", "ork\", "rk\i", "k\it"*
- **Step 2 – Map hashes into indices:** hash each quadgram and find its index via a modulo operation(index = hash % feature size)

Quadgram	Hash value	Index
work	102	2
ork\	555	5
rk\i	880	0
k\it	442	2

  Note: A collision occurred at index two

- **Step 3 – Generate the Feature Vector:**
  We create a vector of length ten and mark the calculated indices.
- **Result:** The path "work\IT" is now represented to the model as [1, 0, 1, 0, 0, 1, 0, 1, 0, 0].

In the real system, we use a much larger vector of size 2048 to minimize collisions, but the principle remains the same. This allows the models to mathematically "see" the structure of the file paths and names without ever needing to understand English or use a lookup table.

This **"Hashing Trick"** is essential for two reasons:

- It allows us to represent an infinite variety of file paths and names using a fixed, manageable number of features.
- The models learn to associate certain quadgrams with HVTs, for example the finance model will associate *"budg"* or *"payr"* with higher priority files to its target focus.

4. Model Selection and Training:

For the classification engine, we selected **Logistic Regression**.

4.1. The Logistic Regression Model Explained:

Logistic Regression is a statistical model used to predict the probability of a binary outcome based on a set of independent variables. In the context of this project, it takes our numerical file features and maps them to a probability score between 0 and 1, where higher values indicate a greater likelihood that the file belongs to a sensitive category (Finance, HR, or IT).

The core of the model is the **Sigmoid function**, which transforms a linear combination of features into a probability curve. The formula is expressed as:

$$P(y = 1) = \frac{1}{1 + e^{-z}}$$

Where z is the weighted sum of the file's features plus a bias term:

$$z = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_n x_n$$

In our implementation, $\{x_i\}_{i=1}^{n}$ represents the features we engineered (like the Recency Score, quadgram hashes and various heuristics), while $\{w_i\}_{i=0}^{n}$ represents the learned weights.

If the resulting z value is large and positive, the probability approaches 1 (High Value). On the other hand, if it is large and negative, the probability approaches 0 (Junk). The mathematical simplicity is what allows the C++ payload to perform complex classification using only basic arithmetic and the exponential function.

4.2. Why logistic regression?

While modern deep learning models are often preferred for text tasks, Logistic Regression provided several critical advantages, especially for a security-focused standalone filesystem scanner:

- **Computational Efficiency:** The final "model" is essentially a single vector of weights and a bias term. This makes the inference process mathematically simple (a dot product followed by a sigmoid function), allowing the C++ payload to remain extremely lightweight and fast.
- **Ease of Portability:** Because the model does not require complex graph execution or heavy libraries (like TensorFlow or PyTorch), the learned parameters can be exported as a simple C++ array stored in a header file, ensuring the payload has zero external dependencies.
- **Interpretability:** During model training understanding why a file was flagged is essential for model tuning. Logistic Regression allows us to inspect the weights of specific features (like a certain path hash or a high recency score) to validate the model's logic and improve performance.

4.3. Target Specific Training:

Rather than training one generic model, we trained four specialized classifiers: **Finance**, **HR**, **IT**, and **General**. All models were trained on the same dataset, yet each model had its own set of labels, according to its designated target. This allows the system to perform fine-tuned classification tailored to the unique data distributions, directory structures, and naming conventions of each target.

4.4. Regularization and Metadata Stability:

To handle the high entropy of filesystem metadata, we utilized regularization. This is crucial for preventing the model from "**overfitting**" the training data - preventing it from memorizing specific file paths that exist in the training data. By tuning the regularization parameter, we ensure the model focuses on generalized patterns, allowing it to remain effective when encountering new, unseen folder structures in a real-world environment.

4.5. Addressing Class Imbalance via Penalty Weights:

In any typical filesystem, sensitive documents are statistically rare compared to the vast amount of system "noise". To solve this **extreme class imbalance**, we applied target-specific class weights. By assigning a higher penalty weight to the "signal" class - we force the Logistic Regression engine to prioritize the detection of rare targets.

Note: Both training parameters were chosen for each model via a grid search that maximized F1-score, while retaining a recall score of above 0.8.

4.6. Prioritizing Recall over Nominal Accuracy:

The goal of these optimizations is to **maximize Recall**. In a data exfiltration scenario, missing a critical file (a false negative) is much more costly than occasionally flagging a non-sensitive file (a false positive). These adjustments ensure the model successfully captures critical assets rather than defaulting to "junk" predictions just to maintain a high, but misleading, accuracy score.

5. System Architecture & C++ Implementation

On a high level, our payload is a lightweight standalone system that scans a given directory, extracts and translates the metadata of all present files into numerical features, calculates the scores of each file according to pre-trained weights and tracks the highest rated files for each target(Finance, IT, etc.). The payload is designed to be incorporated into a real-world malware as a subcomponent. This is reflected in its efficiency and Self-contained nature.

5.1. The Python-to-C++ Bridge:

To transition from the model training environment to a deployable binary, we implemented an export pipeline that translates the Python-trained parameters into C++ constants – stored in a standard header file. This ensures that the parameters learned during training are directly accessible to the C++ payload without the need for external configuration files, libraries, or other programs.

Additionally, we export the relevant valuable and junk extension lists associated with each target and store them in the header file.

Figure 2: *Snippet from ModelWeights.h - Exported Finance model weight and extension mappings.*

```
const double FINANCE_MODEL_BIAS = -4.5145143977;

// recency score - size_logged - valuable_ext -  junk_ext - name_len - ...
const std::vector<double> FINANCE_MODEL_WEIGHTS = {
2.5251071906, 0.0564812350, 0.6183603489, -0.1007858058, -0.0122299967, ...

const std::unordered_set<std::wstring> FINANCE_HIGH_VAL_EXTS =
{L".xlsx", L".xls", L".csv", L".pdf", L".accp", L".pptx"};

const std::unordered_set<std::wstring> FINANCE_JUNK_EXTS =
{L".class", L".java", L".cpp", L".obj", L".dll", L".exe", L".pyc", ...
```

5.2. Traversing the filesystem:

Using the *std::filesystem* library, the scanner performs a recursive walk through the target directory structure. The traversal is designed to be robust - it gracefully handles system errors, permission-denied folders, and symbolic links (directories, sockets, etc.) without halting the process.

5.3. Extracting metadata on the fly:

Continuing the use of *std::filesystem*, we extract the raw attributes from the files. This includes the file path, name, extension, physical size, and the last modification timestamp. We perform necessary normalizations, such as converting the timestamp into a usable format. The extracted data of each file is immediately passed over to the scoring engine, ensuring that data is processed on the fly with zero disk-write usage or delays.

5.4. Feature Engineering in C++:

To ensure the C++ payload produces the exact same results as the Python environment, we implemented a feature transformation module that mimics the process present in python. This module replicates two critical processes:

- **Sliding Window 4-Grams:**
  The scanner extracts character-level quadgrams from file paths and names using the same sliding-window logic as the Python training script.
- **MurmurHash3 Implementation:**
  We implemented the MurmurHash3 algorithm in C++ to match the *Scikit-Learn* HashingVectorizer. By using an identical hash seed and mapping results to the same fixed-size feature vector, we guarantee that a file scanned on a target system produces the exact numerical representation the model was trained on.

5.5. Inference and Scoring:

The final step is the calculation of the logistic probability. The engine computes the dot product of the file's features and the pre-trained model weights, adds the bias term, and passes the result through a native sigmoid function:

$$P(file) = \frac{1}{1 + e^{-(\sum weight \cdot feature + bias)}}$$

This produces a score between 0 and 1. The payload uses these scores to rank files within a priority queue, which are then able to be exploited by an exfiltration module present in the theoretical real-world malware system. The scoring process is repeated for all 4 trained models, each with their own set of pre-trained weights and priority queue.

6. Results & Evaluation

6.1. Model Performance:

The predictive success of the model was evaluated using a hold-out test set of 1,030 files. Given the extreme class imbalance, where target files (Label 1) often represent less than 5% of the data, **standard "Accuracy" is a deceptive metric**. Instead, we focused on the Recall of the minority class to ensure that critical sensitive assets are not missed during a scan, while retaining an adequate degree of precision. The results demonstrate that the class weighting strategy forced the model to prioritize target detection over nominal accuracy:

Focus	Accuracy	Precision	Recall	F1-Score	Weight Multiplier	Model Bias
Finance	98.1%	**0.67**	**0.89**	**0.76**	4.0	-4.5
HR	97.5%	**0.71**	**0.87**	**0.78**	4.0	-5.5
IT	97.5%	**0.80**	**0.82**	**0.81**	12.0	-3.5
General	94.3%	**0.82**	**0.80**	**0.81**	2.0	-4.9

The model achieved high recall in the Finance (0.89) and HR (0.87) categories, successfully prioritizing the capture of sensitive documents over absolute precision. To address the extreme scarcity of IT secrets, an aggressive 12.0 weight multiplier was implemented, resulting in a strong 0.82 recall for critical technical assets like private keys and database credentials. These specialized focuses are reinforced by the General model's 0.81 F1-score, which serves as a robust catch-all for high-value files vs noise.

The consistently low model Bias values across all models (ranging from -3.5 to -5.5) demonstrate that the models have learned a "skeptical" baseline. The system defaults to assuming a file is noise unless it provides strong evidence of relevance through keywords, extensions, and the Recency Score.

6.2. Qualitative Results:

To validate the C++ implementation, the scanner was deployed against a test suite of 3,002 files housed in a simulated corporate environment. This directory mirrors a realistic filesystem, containing departmental folders (Finance, HR, IT), user directories, and significant technical noise.

Snippet of Payload results:

```
ANALYZING TARGET: FINANCE

--- TOP 5 FILES ---
Score: 0.9469 | D:\MLDATA\Users\kimberly\OneDrive - Corp\Finance\Forecasts\FY2023_Corp_Budget_v6.xlsx
Score: 0.9368 | D:\MLDATA\Users\kimberly\OneDrive -Corp\Finance\Forecasts\Forecast_Model_FINAL_v5.xlsm
Score: 0.9205 | D:\MLDATA\Users\kmorris\Documents\Finance\Reporting\HR_Forecast_Sep2024.xlsx
Score: 0.9134 | D:\MLDATA\Users\goodjeffrey\Documents\Finance\Reporting\Ops_Forecast_May2025.xlsx
Score: 0.8889 | D:\MLDATA\Users\ybrown\Documents\Finance\Reporting\R&D_Forecast_Sep2024 (1).xlsx

--- BOTTOM 5 FILES ---
Score: 0.0001 | D:\MLDATA\Users\nrodriguez\Documents\misc\old\document_87n1b.mp3
Score: 0.0001 | D:\MLDATA\Users\Dev\Workspace\node_modules\lib_82\sjzweg.js
Score: 0.0001 | D:\MLDATA\Users\Dev\Workspace\node_modules\lib_57\g10946.js
Score: 0.0001 | D:\MLDATA\Users\Dev\Workspace\node_modules\lib_281\h6lc36.js
Score: 0.0001 | D:\MLDATA\Users\Dev\Workspace\node_modules\lib_385\ypex4z.js

ANALYZING TARGET: IT

--- TOP 5 FILES ---
Score: 0.7774 | D:\MLDATA\Users\zphillips\Desktop\.env.production
Score: 0.7476 | D:\MLDATA\Infra\Backups\.env.production
Score: 0.6695 | D:\MLDATA\Users\User_1\AppData\Local\Temp\f8a2c1.key
Score: 0.5581 | D:\MLDATA\Users\emily49\Desktop\.env.production
Score: 0.3435 | D:\MLDATA\Infra\Certificates\.env.production

--- BOTTOM 5 FILES ---
Score: 0.0000 | D:\MLDATA\Users\awilliams\Desktop\Order Granting Motion to Amend Complaint -
AcmeCorp_XYZInc_2010 - ORDER_785.docx
Score: 0.0000 | D:\MLDATA\Users\awilliams\Downloads\Order Granting Motion to Amend Complaint - Smith_Bank of
America_2016.docxScore: 0.0000 | D:\MLDATA\Users\scottjonathan\Desktop\Order Granting Motion to Amend Complaint
- Johnson_GlobalBank_2014.pdf
Score: 0.0000 | D:\MLDATA\Users\marygonzalez\Documents\Legal\Smith_XYZInc_2014_findings_and_conclusions -
ORDER_723.pdf
Score: 0.0000 | D:\MLDATA\Users\kristenmartin\Documents\Legal\Class Notice of Compromise - Johnson_Doe_2008.pdf
```

6.3. Key Findings from Results:

- **Effective Discrimination:**
  The system successfully assigns high scores to relevant targets and extremely low scores to "noise". For example, a budget spreadsheet scored **0.9469**, while technical junk in *node_modules* scored as low as **0.0001**.

- **Specialized Classification:**
  The models are target-specific - the IT focus correctly identifies .env and .key files as high-value (scores **0.7774–0.6695**) while ignoring office documents that a General or Finance model might prioritize.

```
DEBUG SCORE for HR:
Salary_Scales_2025_FINAL.xlsx
  Bias: -5.5945
  Recency (val=0.5318): 2.1422
  Size (val=12.2103):    -1.1031
  Name Length (val=29.0000): -0.2547
  Path Length (val=34.0000): -1.6116
  Path Depth (val=4.0000): -0.4800
  Ext (val=1.0000):     0.9264
  Junk Ext (val=0.0000):  -0.0000
  Name Hash Contrib: 3.2870
  Path Hash Contrib: 6.3647
  FINAL LOGIT (Z):   3.6762
  FINAL PROBABILITY: 0.9753
```

HVT Case Study - HR Focus:

Path(+6.36) and filename(+3.28):
The model correctly identified the files directory and name as a high priority. The path was the primary driver of the score, outweighing the filename itself.
Recency (+2.14): Despite the bias of -5.59, the 2025 timestamp metadata provided a significant positive boost, allowing the model to distinguish this file from outdated files.
Path length(-1.61): The HR model tends to associate relevant files with shorter paths, rather than deep or long paths.

6.4. System Performance:

The payload is broken down into two main phases:

- Directory Traversal and Metadata Extraction
- Feature engineering and inference

We Benchmarked the same test directory of 3,002 files across different storage media – **comparing SSD vs HDD performance**. This reveals how the extraction and Inference phases scale differently.

While the extraction phase is heavily dependent on hardware throughput and disk I/O performance, the inference phase remains computationally stable across both environments:

Metrics (3,002 Files)	**SSD Environment**	**HDD Environment**	**Delta(%)**
Traversal and Extraction	175.61 ms 0.058 ms/file	446.48 ms 0.148 ms/file	+154%
Feature engineering and inference	965.92 ms 0.080 ms/file per target	1013.57 ms 0.084 ms/file per target	+4.9%
Total Execution Time	1,141.53 ms	1,460.05 ms	+27.8%

- **Traversal and Extraction Phase (I/O Bound):**
  Moving the target directory to an HDD resulted in a 154% increase in extraction time. This confirms that the primary bottleneck for the scanner's speed is the read speed of the disk and the overhead of recursive traversal.
- **Inference Phase (CPU Bound):**
  The inference time remained nearly identical across both tests (~1.0s). This is because once the metadata is extracted and vectorized and has been loaded into RAM, the scoring process - consisting of dot-product and sigmoid calculations - is purely a CPU-dependent task.
- **Conclusion:**
  Even on slower legacy hardware (HDD), the scanner identifies and ranks over 3,000 files in under 1.5 seconds. This demonstrates that the "intelligence" added by the ML models does not create a significant footprint, even when the I/O environment is suboptimal. This is an adequate result showing the performance gained via the compiled low-level C++ payload.

7. Obstacles, Struggles & Engineering Challenges:

The transition from a Python training and research environment to a low-level C++ payload presented several fundamental engineering hurdles. These challenges highlighted the gap between theoretical machine learning and the harsh constraints of real-world systems programming:

Building The Dataset and Working with Synthetic Data:

Expectedly, constructing the dataset for our project was the most complex part of our work. Piecing together a dataset from various sources, filling in missing pieces of data (paths, timestamps, etc.), labeling data using hard-coded logic - Resulted in a seemingly realistic representation of data for our model.

And yet, taking a closer look at the records makes it apparent that substantial portions of the data are not authentic human generated records, introducing an “Uncanny valley” effect. In theory, using data from previously breached networks and having analysts label the records would give the system a massive real-world performance boost.

Defining a High Value target:

One of the most significant non-technical struggles was defining a universal standard for what constitutes a "High-Value Target" (HVT). We quickly realized that the definition of a sensitive file is highly context-dependent, a file that is critical for a Finance department is often complete noise for an IT administrator. This led to the realization that a single "General" model was insufficient.

We pivoted our strategy to develop specialized classifiers for each target focus, acknowledging that the model’s "intuition" must shift based on the specific targets we are after. This challenge extended into the data labeling phase, where defining concrete criteria for an HVT results in a harsh cut-off that struggled to capture the nuances of human-generated filesystem data.

Ensuring consistency between the environments:

A significant obstacle was ensuring that the C++ scanner's feature extraction logic remained 100% identical to Pythons. Any deviation in how files were processed would render the model's "intelligence" useless. We faced an elevated risk of "feature drift" due to differences in how Python and C++ handle string encoding (UTF-8 vs. Wide Strings), signed/unsigned integer arithmetic, and the internal mechanics of the MurmurHash3 algorithm. A single mismatch in a hash calculation meant the scanner would apply weights to the wrong features, leading to entirely incorrect scoring.

8. Future Work & Improvements

The current implementation is a functional proof-of-concept for metadata-driven exfiltration, but several features could be implemented to transform our system into a production ready tool:

- **Scanner Stealth and Performance Optimization**:

  Future iterations will focus on enhancing the operational security of the C++ binary. This includes implementing advanced stealth features such as CPU throttling to bypass system level anti-virus software. Additionally, we could explore parallelization of the filesystem traversal. By utilizing multi-threading, the scanner could drastically reduce its "dwell time" on a target machine, though this must be carefully balanced against the increased CPU signature that parallel processing creates.

- **Feature Engineering and Architectural Exploration**:

  We could increase our feature space by exploring areas such as file entropy(could help detect non-human generated files). Additionally, moving beyond Logistic Regression to more complex architectures - such as Gradient Boosted Decision Trees or lightweight Neural Networks - might allow the system to capture more non-linear relationships between file paths and their perceived importance.

- **Hybrid Content-Metadata Analysis:**

  To bridge the gap between speed and precision, a hybrid analysis mode is a logical next step. In this configuration, the metadata engine would function as a first-stage filter to identify high-probability candidates, which are then passed to a secondary "shallow" content scanner. This second stage would verify the files' internal structure (e.g., checking for specific headers or analyzing the files' contents) only for the most critical targets, ensuring high precision without the performance penalty of a full-drive content scan.

- **Real-World Tuning and Domain Adaptation:**

  The system would benefit significantly from tuning against authentic, non-synthetic datasets. Training on diverse, un-augmented drives from various industries would help the model move past the "Uncanny Valley" of synthetic patterns and adapt to the organic entropy of real-world corporate filesystems. If targeting the same network multiple times, we can leverage the data gained from previous breaches to further tune our model to that target network – improving the performance for networks that were breached multiple times.

9. Conclusion

This project successfully demonstrated that a **metadata-centric machine learning approach** is a viable and stealthy alternative to traditional methods. By focusing on structural heuristics, temporal data, and text-hashing, we developed a system capable of identifying high-value targets while remaining lightweight enough to bypass typical network monitoring. Building this pipeline from the ground up served as a practical lesson in the "dirty" side of Data Science, forcing me to bridge the gap between theoretical knowledge and the harsh constraints of real-world issues and low-level programming.

Technical Insights & Personal Growth

The journey from data scraping to a compiled C++ binary highlighted several critical realities of real-world engineering:

- **The Reality of Data Sourcing:** I learned that the most complex part of a machine learning project is often the data itself. Constructing a dataset from old disk images, GitHub repositories, and other various sources taught me the difficulty of simulating authentic human-generated data.

- **Making sure everything matches:** Ensuring environment consistency was a significant hurdle. I discovered how easily minute differences in string encoding, integer arithmetic or implementation can lead to "feature drift," where a single mismatch renders the model useless in a real-world payload.

- **Accuracy isn't everything:** This project shifted my perspective on model performance. In a data exfiltration scenario, missing a critical file is far more costly than occasionally flagging noise. **Learning to Prioritize Recall and model "Vibe"** rather than just chasing high nominal accuracy was a vital lesson in engineering for specific outcomes.

- **The Beauty of Simple Models:** Testing the model made me appreciate the elegance of Logistic Regression. It showed that complex neural networks are not always necessary. A simple dot-product and sigmoid function can provide adequate performance with a negligible CPU and memory footprint.

Final Reflection:

Ultimately, this project was about more than just ranking files. It was an exercise in building a functional, end-to-end system that leverages ML to help solve a known problem. I am walking away with a much deeper understanding of how to leverage the inherent patterns of various types of metadata to create tools that are accurate, efficient, and deployable in the real-world. To me, it is apparent that **the integration of Data Science** is not merely an enhancement, but the **next logical and inevitable step in modern cybersecurity**.